Université d'Ottawa
Faculté de génie

École de science informatique
et de génie électrique

uOttawa

L'Université canadienne
Canada's university

University of Ottawa
Faculty of Engineering

School of Electrical Engineering
and Computer Science

Assignment 1 (3.75% – 50 points)

CSI2110 (Fall 2026)

Due: 11:59PM, Friday, October 9, 2026

Late assignment policy: 1min–24hs late are accepted with 30% off; no assignments accepted after 24hs late.

Question 1. Asymptotic notations **[20 points = 4+4+4+4+4 points]**

Decide if each of the following statements is **true or false** and give a **proof**. For a true statement you need to identify the values for the constants c and n_0 as used in the definitions of big-O, Ω and Θ and show the corresponding inequalities. For a false statement, you need to justify/prove why finding those constants is impossible. Unless otherwise specified, all logarithms are base 2.

(a) $21\log(n^3) + 10(\log n)^2 + \sqrt{n}$ is $O(\sqrt{n})$

(b) $(3n^2 - n + 2)^2$ is $\Omega(n^2)$

(c) $\log_{10}(21n + n^{10})$ is $\Theta(\log n)$

(d) $2^{2n} + n$ is $O(2^n)$

(e) $\sum_{i=0}^{n} \frac{2^i}{2^n} = \frac{1}{2^n} + \frac{2}{2^n} + \frac{4}{2^n} + \cdots + \frac{2^n}{2^n}$ is $O(1)$

Question 2. Asymptotic algorithm analysis **[10 points = 5+5 points]**

The following algorithm returns the index of the first row whose entries are all zero, or `-1` if no such row exists. Assume that `A` is a non-null $n \times n$ integer array with $n \geq 2$, and that all rows are non-null. Express your bounds in terms of n, the number of rows.

Note: Give the **tightest big-O bound**. For example, if you can prove that $f(n)$ is $O(n)$, and that $f(n)$ is $O(n^2)$, choose the tighter upper bound, i.e., $f(n)$ is $O(n)$.

```
1  public static int findZeroRow(int[][] A) {
2      int n = A.length;
3      for (int i = 0; i < n; i++) {
4          int j = 0;
5          while (j < n && A[i][j] == 0) {
6              j++;
7          }
8          if (j == n) {
9              return i;
10         }
11     }
12     return -1;
13 }
```

(a) Give a big-Oh bound for $T(n)$, the **worst-case** running time of this algorithm. Describe a family of inputs, not just one example, that gives the worst case.

(b) Give a big-Oh bound for $B(n)$, the **best-case** running time of this algorithm. Describe a family of inputs, not just one example, that gives the best case.

Question 3. Stack, Queue, and Deque **[20 points = 8+2+10 points]**

A simplified print job manager maintains a waiting line of jobs and one printer. The printer can process at most one job at a time. Initially, the waiting line is empty and the printer is idle. All submitted jobs are non-null.

Throughout this question, assume that the data structure has sufficient capacity. No overflow checks or resizing are required.

(a) Choose the most suitable ADT covered in class and give pseudocode or Java code for the following four operations. State your choice of ADT and any additional variables you use.

1. `Submit(Job p, boolean urgent)`: Add job `p` to the front of the waiting line if `urgent` is true; otherwise, add it to the back. This does not interrupt the job currently being processed.
2. `StartNext()`: If the printer is idle and the waiting line is nonempty, remove the first waiting job and make it the current job. Otherwise, do nothing.
3. `Postpone()`: If there is a current job, move it to the back of the waiting line and make the printer idle. Otherwise, do nothing.
4. `CancelLast(int k)`: Remove the last k waiting jobs. Assume $k \geq 0$. If fewer than k jobs are waiting, remove all waiting jobs. Do not affect the current job.

Use the operations of the chosen ADT without implementing the ADT itself. You may use an additional reference to store the current job. Simplicity counts.

(b) Execute the following sequence and show the output of each call to `PrintWaiting()`. This operation prints the waiting jobs from front to back, excludes the current job, and does not modify the waiting line. The letters `A` through `F` identify distinct jobs.

```
Submit(A, false);
Submit(B, false);
Submit(C, true);
PrintWaiting();
StartNext();
Submit(D, true);
Submit(E, false);
PrintWaiting();
Postpone();
StartNext();
PrintWaiting();
Submit(F, true);
CancelLast(2);
PrintWaiting();
```

(c) We would like to improve the implementation of `CancelLast(k)` so that it does not need to remove the waiting jobs one at a time. Consider a new ADT, called `JobLine`, that stores the waiting jobs and supports the following four operations:

Operation	Description
`addFirst(Job p)`	Add job `p` to the front of the waiting line.
`addLast(Job p)`	Add job `p` to the back of the waiting line.
`removeFirst()`	Remove and return the first waiting job, or return `null` if the waiting line is empty.
`cancelLast(int k)`	Remove the last k waiting jobs. If fewer than k jobs are waiting, remove all waiting jobs. This operation does not return the removed jobs.

Example. Initially, the waiting line is (A, B, C, D, E), from front to back. Execute the following operations in order:

1. `cancelLast(2)` leaves (A, B, C).
2. `addLast(F)` gives (A, B, C, F).
3. `addFirst(G)` gives (G, A, B, C, F).
4. `removeFirst()` returns `G`, leaving (A, B, C, F).
5. `cancelLast(10)` leaves an empty waiting line.

Design a **concrete data structure** implementing this ADT such that **each operation runs in $O(1)$ worst-case time**, independent of both the number of waiting jobs and the value of k. The relative order of the remaining jobs must be preserved.

(i) Describe your underlying data structure and the variables it maintains. Give their initial values for an empty waiting line.

(ii) Give pseudocode or Java code for `addFirst`, `addLast`, `removeFirst`, and `cancelLast`. Implement these operations using your underlying data structure.

(iii) Briefly justify the correctness and worst-case running time of your implementation for `cancelLast (int k)`.